

Linux Fundamentals — Instructor Guide

Every challenge, answer, and accepted variant · generated 2026-08-18 from the live pack data

Quick reference

Set your own site URL and class password before handing this out. Admin handle: `warden` .

Handles: 3–20 characters, letters/numbers/underscore/hyphen, **claimed once only**.

40 challenges · 960 points total · flags look like `FLAG{ABCD2345EFGH}` and are **different for every student**.

How the difficulty fades — Act I briefs show the whole command, and the free hint repeats it. From Act II the brief names the tool but not its flags or arguments, and the exact line moves into a hint that costs XP. By the last act the brief states only the objective. A student who never buys a hint has genuinely recalled every command. Hint spend is visible in the instructor dashboard, so it reads as a difficulty signal rather than as cheating.

If a student says "it's not working"

Symptom	Cause & what to say
"My flag is rejected"	Flags are per-student — a classmate's flag never validates. Tell them to click the green flag in the terminal to copy it exactly, rather than retyping.
"Handle already claimed"	Someone took it, or they registered before and cleared their browser. Pick a new handle. (To free one, delete that player in Netlify Blobs.)
"I ran the command, nothing happened"	Check the blue path in their prompt — they may be in a different directory. Most challenges accept short or full paths, but the file must exist from where they stand. <code>cd ~</code> returns home.
"It says command not found"	If it's a real command we don't simulate (<code>top</code> , <code>nano</code>), the game says so explicitly. Genuine typos say "command not found".
"I don't know which command to use"	Intended from Act II on. Point them at the free hint first, then the Reference tab. The costed hint is the safety net, not a failure.
"The act is locked"	Each act opens after solving all but one of the previous act's challenges.
"Nothing printed"	Correct for <code>cd</code> and redirects. The game prints "(no output — in a shell, silence usually means success)".
Lost all progress	Progress is tied to the browser token. Same browser = resumes automatically. Different machine = new handle needed.

Act structure & unlocking

Act	Teaches	Challenges	Points	Unlock
Act I: Navigation & Files	Standing your ground, inspecting files, and shell wildcards	10	150	Always open
Act II: Text Streams & Pipes	Filtering, counting, sorting, and connecting data streams	10	260	Solve 8 of 10 in the prior act
Act III: Operations & Permissions	Creating, copying, removing, permissions, and sudo elevation	10	235	Solve 8 of 10 in the prior act

Act IV: Advanced Shell & Scripting	Find, sed, awk, exit codes, conditionals, and variables	10	315	Solve 8 of 10 in the prior act
---	---	----	-----	--------------------------------

Students may skip **one** challenge per act and still advance.

Act I: Navigation & Files

Standing your ground, inspecting files, and shell wildcards

Present Working Directory

10 pts · I1-pwd · OK

Task: Run `pwd` to display your current directory position in the filesystem tree.

Answer: `pwd`

Completion: Auto-completes on: commandMatches

Hint 1 (free): Type `pwd` and press Enter.

Also accepted: `/bin/pwd`

Directory Listing

10 pts · I1-ls · OK

Task: Use `ls` to list the files and folders inside your current directory.

Answer: `ls`

Completion: Auto-completes on: commandMatches

Hint 1 (free): Type `ls .`

Also accepted: `ls .`, `ls ./`

Reveal Hidden Dotfiles

15 pts · I1-ls-la · OK

Task: In Linux, filenames beginning with a dot (`.`) are hidden by default. Use `ls -la` to inspect your home directory and view hidden configuration files.

Answer: `ls -la` then `ls -al`

Completion: Auto-completes on: commandMatches

Hint 1 (free): Run `ls -la` or `ls -al`.

Also accepted: `ls -al`, `ls -a -l`

Changing Directories

15 pts · I1-cd · OK

Task: Use `cd Documents` to move into the Documents directory.

Answer: `cd Documents`

Completion: Auto-completes on: cwdls

Hint 1 (free): Type `cd Documents`.

Also accepted: `cd ./Documents`, `cd /home/student/Documents`

Climbing Up (..)

15 pts · l1-cd-parent · OK

Task: The special symbol `..` represents the parent directory. From `Documents`, use `cd ..` to return to your home directory.

Answer: `cd ..`

Completion: Auto-completes on: `cwdls`

Hint 1 (free): Type `cd ..`.

Also accepted: `cd ../`, `cd ~`, `cd /home/student`

Display File Content

15 pts · l1-cat · OK

Task: Print the contents of `Documents/notes.txt` to the screen using `cat`.

Answer: `cat Documents/notes.txt`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): Run `cat Documents/notes.txt`.

Also accepted: `cat ./Documents/notes.txt`, `cat /home/student/Documents/notes.txt`

First Few Lines (head -n)

15 pts · l1-head · OK

Task: `Documents/data.csv` is long. Show only its first 3 lines using `head`.

Answer: `head -n 3 Documents/data.csv`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): Run `head -n 3 Documents/data.csv`.

Also accepted: `head -3 Documents/data.csv`

Last Few Lines (tail -n)

15 pts · l1-tail · OK

Task: Show only the last 2 lines of `Documents/data.csv` using `tail`.

Answer: `tail -n 2 Documents/data.csv`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): Run `tail -n 2 Documents/data.csv`.

Also accepted: `tail -2 Documents/data.csv`

Wildcard Globbing (*)

20 pts · l1-glob · OK

Task: The shell expands wildcard patterns before the command runs. List every `.js` file inside `projects/web` with a single `*` pattern instead of typing the filenames.

Answer: `ls projects/web/*.js`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): Run `ls projects/web/*.js`.

Also accepted: `ls ./projects/web/*.js`

Multiple Wildcards

20 pts · l1-glob-doc · OK

Task: List every `.txt` file in `Documents` with one wildcard pattern.

Answer: `ls Documents/*.txt`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): Run `ls Documents/*.txt`.

Also accepted: `ls ./Documents/*.txt`

Act II: Text Streams & Pipes

Filtering, counting, sorting, and connecting data streams

Search Patterns with grep

20 pts · l2-grep · OK

Task: Find every line that contains the word `active` in `Documents/data.csv`. Use `grep`.

Answer: `grep active Documents/data.csv`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `grep` takes the pattern first, then the file to search.

Hint 2 (–10 XP): Run `grep active Documents/data.csv`.

Also accepted: `grep "active" Documents/data.csv`

Case-Insensitive Grep (-i)

20 pts · l2-grep-i · OK

Task: `logs/app_errors.log` spells errors three ways: `ERROR`, `Error` and `error`. Find all of them with one `grep` search that ignores letter case.

Answer: `grep -i "error" logs/app_errors.log`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): One short flag makes `grep` ignore case.

Hint 2 (–10 XP): Run `grep -i "error" logs/app_errors.log`.

Also accepted: `grep -i "error" logs/app_errors.log`

Inverting Matches (-v)

25 pts · l2-grep-v · OK

Task: `logs/app_errors.log` is mostly `INFO` noise. Use `grep` to show every line EXCEPT the ones containing `INFO`.

Answer: `grep -v "INFO" logs/app_errors.log`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): One flag inverts a `grep` match, so it prints the lines that do NOT match.

Hint 2 (–10 XP): Run `grep -v "INFO" logs/app_errors.log`.

Also accepted: `grep -v "INFO" logs/app_errors.log`

Line Counting (wc -l)

20 pts · l2-wc-l · OK

Task: Count how many records `Documents/server_access.log` holds. Use `wc`.

Answer: `wc -l Documents/server_access.log`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `wc` counts lines, words and bytes. One flag limits it to lines.

Hint 2 (–10 XP): Run `wc -l Documents/server_access.log`.

Also accepted: `wc -l ./Documents/server_access.log`

Sorting Data (sort)

25 pts · l2-sort · OK

Task: Documents/data.csv is comma separated. Sort it by the 4th field, salary, in numeric order. `sort` needs to know the separator, which field, and that the field is a number.

Answer: `sort -t, -k4 -n Documents/data.csv`

Completion: Auto-completes on: commandMatches

Hint 1 (free): Three flags: one sets the field separator, one picks the field number, one forces numeric order.

Hint 2 (-10 XP): Run `sort -t, -k4 -n Documents/data.csv`.

Also accepted: `sort -t, -n -k4 Documents/data.csv`

Field Extraction (cut)

25 pts · l2-cut · OK

Task: Documents/data.csv is comma separated. Print only field 2 (name) and field 4 (salary). Use `cut`.

Answer: `cut -d, -f2,4 Documents/data.csv`

Completion: Auto-completes on: commandMatches

Hint 1 (free): One flag sets the delimiter, another picks the fields. Separate the field numbers with a comma.

Hint 2 (-10 XP): Run `cut -d, -f2,4 Documents/data.csv`.

Also accepted: `cut -d" ," -f2,4 Documents/data.csv`

The Pipeline (grep | wc -l)

30 pts · l2-pipe-wc · OK

Task: Count how many requests in Documents/server_access.log returned status 401. Do it in a single line: search the log, then feed the matching lines straight into a counter.

Answer: `wc -l then grep "401" Documents/server_access.log | wc -l`

Completion: Auto-completes on: commandMatches

Hint 1 (free): The `|` operator sends one command's output into the next one. `grep` finds the lines, `wc -l` counts them.

Hint 2 (-10 XP): Run `grep "401" Documents/server_access.log | wc -l`.

Also accepted: `grep "401" Documents/server_access.log | wc -l`

Output Redirection (>)

30 pts · l2-redirect-out · OK

Task: Save every ERROR line from logs/app_errors.log into a new file at /tmp/critical_errors.txt. Nothing should print to the screen.

Answer: `grep "ERROR" logs/app_errors.log > /tmp/critical_errors.txt`

Completion: Auto-completes on: allOf

Hint 1 (free): `>` sends a command's output into a file instead of the screen.

Hint 2 (-10 XP): Run `grep "ERROR" logs/app_errors.log > /tmp/critical_errors.txt`.

Also accepted: `grep ERROR logs/app_errors.log > /tmp/critical_errors.txt`

Append Redirection (>>)

30 pts · I2-redirect-append · OK

Task: Add the line `Audit log verified` to the END of `/tmp/audit_trail.log` without erasing what is already in it.

Answer: `echo "Audit log verified" >> /tmp/audit_trail.log`

Completion: Auto-completes on: allOf

Hint 1 (free): `>` overwrites a file. A doubled version of the same operator appends instead. Use `echo` to produce the text.

Hint 2 (-10 XP): Run `echo "Audit log verified" >> /tmp/audit_trail.log`.

Also accepted: `echo Audit log verified >> /tmp/audit_trail.log`

Split Output (tee)

35 pts · I2-tee · OK

Task: Show every line containing `Security` from `Documents/data.csv` on screen AND save those same lines to `/tmp/security_team.txt`, in one command.

Answer: `grep "Security" Documents/data.csv | tee /tmp/security_team.txt`

Completion: Auto-completes on: allOf

Hint 1 (free): `tee` writes its input to a file and passes it along to the screen at the same time.

Hint 2 (-10 XP): Run `grep "Security" Documents/data.csv | tee /tmp/security_team.txt`.

Also accepted: `grep Security Documents/data.csv | tee /tmp/security_team.txt`

Act III: Operations & Permissions

Creating, copying, removing, permissions, and sudo elevation

Create Empty Files (touch)

20 pts · l3-touch · OK

Task: Create an empty file at `/tmp/new_file.txt`.

Answer: `touch /tmp/new_file.txt`

Completion: Auto-completes on: `fileExists`

Hint 1 (free): `touch` creates an empty file, or updates the timestamp of one that exists.

Hint 2 (-10 XP): Run `touch /tmp/new_file.txt`.

Make Directories (mkdir -p)

20 pts · l3-mkdir · OK

Task: Create the directory `projects/api`. Use the flag that also creates any missing parent directory, so the command works whether or not `projects` already exists.

Answer: `mkdir -p projects/api`

Completion: Auto-completes on: `dirExists`

Hint 1 (free): `mkdir` makes directories. One flag creates missing parents and stays quiet if the directory is already there.

Hint 2 (-10 XP): Run `mkdir -p projects/api`.

Also accepted: `mkdir projects/api`

Copy Files (cp)

20 pts · l3-cp · OK

Task: Make a backup copy of `Documents/notes.txt` at `/tmp/notes_backup.txt`.

Answer: `cp Documents/notes.txt /tmp/notes_backup.txt`

Completion: Auto-completes on: `fileExists`

Hint 1 (free): `cp` takes the source path first, then the destination path.

Hint 2 (-10 XP): Run `cp Documents/notes.txt /tmp/notes_backup.txt`.

Also accepted: `cp ./Documents/notes.txt /tmp/notes_backup.txt`

Recursive Copy (cp -r)

25 pts · l3-cp-r · OK

Task: Copy the whole `projects/web` directory, with everything inside it, to `/tmp/web_backup`.

Answer: `cp -r projects/web /tmp/web_backup`

Completion: Auto-completes on: `dirExists`

Hint 1 (free): `cp` refuses a directory until you add the recursive flag.

Hint 2 (-10 XP): Run `cp -r projects/web /tmp/web_backup`.

Also accepted: `cp -R projects/web /tmp/web_backup`

Move & Rename (mv)

20 pts · l3-mv · OK

Task: Rename `projects/scripts/deploy.py` to `deploy_prod.py`. Leave it in the same directory.

Answer: `mv projects/scripts/deploy.py projects/scripts/deploy_prod.py`

Completion: Auto-completes on: `fileExists`

Hint 1 (free): `mv` both moves and renames. Give it the old path, then the new path.

Hint 2 (-10 XP): Run `mv projects/scripts/deploy.py projects/scripts/deploy_prod.py`.

Also accepted: `mv ./projects/scripts/deploy.py ./projects/scripts/deploy_prod.py`

Remove Files (rm)

20 pts · l3-rm · OK

Task: Delete the temporary file `/tmp/scratch.txt`.

Answer: `rm /tmp/scratch.txt`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `rm` removes a file. There is no undo and no recycle bin.

Hint 2 (-10 XP): Run `rm /tmp/scratch.txt`.

Also accepted: `rm -f /tmp/scratch.txt`

Permissions (chmod octal)

30 pts · l3-chmod-numeric · OK

Task: Lock down `Documents/notes.txt` so the owner can read and write it and nobody else can do anything at all. Set the octal mode.

Answer: `chmod 600 Documents/notes.txt`

Completion: Auto-completes on: `fileHasMode`

Hint 1 (free): Read is 4, write is 2, execute is 1. Add them up per column: owner, group, other.

Hint 2 (-10 XP): Run `chmod 600 Documents/notes.txt`.

Also accepted: `chmod 600 ./Documents/notes.txt`

Make Executable (chmod +x)

25 pts · l3-chmod-symbolic · OK

Task: `projects/scripts/deploy.py` cannot be run yet. Add execute permission to it.

Answer: `chmod +x projects/scripts/deploy.py`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `chmod` also accepts symbolic modes: a `+` or `-`, then a letter for the permission.

Hint 2 (-10 XP): Run `chmod +x projects/scripts/deploy.py`.

Also accepted: `chmod u+x projects/scripts/deploy.py`

Inspect Inodes & Timestamps (stat)

20 pts · l3-stat · OK

Task: Show the size, permissions, inode number and timestamps of `Documents/notes.txt`. `ls -l` does not show all of that.

Answer: `stat Documents/notes.txt`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): One command reports a file's full metadata block.

Hint 2 (–10 XP): Run `stat Documents/notes.txt`.

Also accepted: `stat ./Documents/notes.txt`

Superuser Elevation (sudo)

35 pts · l3-sudo-shadow · OK

Task: `/etc/shadow` holds the account password hashes, and only root may read it, so `cat /etc/shadow` is denied. Read the file anyway.

Answer: `sudo cat /etc/shadow`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): One command runs a single other command as a different user, normally root.

Hint 2 (–10 XP): Run `sudo cat /etc/shadow`.

Instructor note: The brief deliberately shows the command that FAILS. Elevation is the lesson.

Act IV: Advanced Shell & Scripting

Find, sed, awk, exit codes, conditionals, and variables

Finding Files by Name (find)

30 pts · I4-find-name · OK

Task: Shell scripts are scattered somewhere under your home directory. List every file whose name ends in `.sh`, at any depth below the current directory.

Answer: `find . -name "*.sh"`

Completion: Auto-completes on: commandMatches

Hint 1 (free): `find` walks a whole directory tree. It can filter on the filename, and the pattern should be quoted so the shell does not expand it first.

Hint 2 (–15 XP): Run `find . -name "*.sh"`.

Also accepted: `find . -name '*.sh'`

Filtering by Type (find -type f)

30 pts · I4-find-type · OK

Task: List every regular file underneath `projects` — the files themselves, not the directories that hold them.

Answer: `find projects -type f`

Completion: Auto-completes on: commandMatches

Hint 1 (free): `find` can filter by entry type. The letter for a regular file is `f`.

Hint 2 (–15 XP): Run `find projects -type f`.

Also accepted: `find ./projects -type f`

Stream Editing (sed s///)

35 pts · I4-sed-replace · OK

Task: Print `Documents/data.csv` with every occurrence of `Engineering` replaced by `Tech`. The file on disk must not change.

Answer: `sed "s/Engineering/Tech/g" Documents/data.csv`

Completion: Auto-completes on: commandMatches

Hint 1 (free): `sed` edits a stream as it passes through. Its substitute script has the form `s/old/new/g`.

Hint 2 (–15 XP): Run `sed "s/Engineering/Tech/g" Documents/data.csv`.

Also accepted: `sed 's/Engineering/Tech/g' Documents/data.csv`

Pattern Scanning (awk)

35 pts · I4-awk-column · OK

Task: `/etc/passwd` is colon separated. Print only the username (field 1) and the home directory (field 6) for every account on the system.

Answer: `awk -F: '{print $1, $6}' /etc/passwd`

Completion: Auto-completes on: commandMatches

Hint 1 (free): `awk` splits each line into numbered fields. One flag sets the separator, and `$1` refers to the first field.

Hint 2 (–15 XP): Run `awk -F: '{print $1, $6}' /etc/passwd`.

Also accepted: `awk -F: "{print $1, $6}" /etc/passwd`

Variable Expansion & Quotes

25 pts · I4-var-quotes · OK

Task: Print the value of your `HOME` variable. Quote it in the way that still lets the shell expand it.

Answer: `echo "$HOME"`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): Double quotes allow variable expansion. Single quotes suppress it and print the name literally.

Hint 2 (-15 XP): Run `echo "$HOME"`.

Exit Codes (\$?)

30 pts · I4-exit-status · OK

Task: Every program returns an exit code: 0 means success, anything else means failure. Search `/etc/passwd` for `root`, then print the exit code that the search returned — both on one line.

Answer: `grep "root" /etc/passwd; echo $?`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `$?` holds the exit code of the previous command, and `;` separates two commands on one line.

Hint 2 (-15 XP): Run `grep "root" /etc/passwd; echo $?`.

Also accepted: `grep root /etc/passwd; echo $?`

Conditional AND (&&)

30 pts · I4-list-and · OK

Task: On one line, create the directory `/tmp/test_dir` and then create the file `/tmp/test_dir/file.txt` inside it — but only attempt the file if the directory step succeeded.

Answer: `mkdir -p /tmp/test_dir && touch /tmp/test_dir/file.txt`

Completion: Auto-completes on: `fileExists`

Hint 1 (free): `&&` runs the second command only when the first one exits with success.

Hint 2 (-15 XP): Run `mkdir -p /tmp/test_dir && touch /tmp/test_dir/file.txt`.

Conditional OR (||)

30 pts · I4-list-or · OK

Task: On one line, try to read `/non_existent_file`, and if that attempt fails, print the message `File was missing`.

Answer: `cat /non_existent_file || echo "File was missing"`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `||` runs the second command only when the first one fails.

Hint 2 (-15 XP): Run `cat /non_existent_file || echo "File was missing"`.

Also accepted: `cat /non_existent_file || echo 'File was missing'`

Compare Files (diff)

30 pts · l4-diff · OK

Task: Compare `projects/web/index.html` against `projects/web/style.css` line by line, in the unified format that patch tools read.

Answer: `diff -u projects/web/index.html projects/web/style.css`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): `diff` compares two files. One flag selects unified output.

Hint 2 (-15 XP): Run `diff -u projects/web/index.html projects/web/style.css`.

Also accepted: `diff -u ./projects/web/index.html ./projects/web/style.css`

Shell Mastery (history)

40 pts · l4-history · OK

Task: Show every command you have typed in this session.

Answer: `history`

Completion: Auto-completes on: `commandMatches`

Hint 1 (free): The shell keeps a numbered list of what you have run. One word prints it.

Hint 2 (-15 XP): Run `history`.

Accepted command variants

Auto-completing challenges accept the natural variations a beginner types, so a correct command is never silently ignored:

- Optional quotes around a path
- Path prefixes: `./Documents`, `~/Documents`, or the full absolute path
- Short paths when already inside the folder
- Flag spacing: `head -n 5`, `head -n5`, `head -5`
- Windows: forward or back slashes, any capitalization (`md5` or `MD5`)
- Trailing spaces are ignored everywhere

Teaching notes

- **The Coach** (toggle in the terminal title bar) explains every command and error in plain language. Explanations fade after the second use of a command. Leave it ON for beginners.
- **Reference tab** holds a manual page for every command, and tells students the real-shell equivalent. Point stuck students there before they buy a hint.
- **Unsimulated tools** reply with what they are, rather than a bare "command not found".
- **Hints:** the first is free and conceptual; the one that gives the exact command costs XP. Hint use is visible in the instructor dashboard as a signal of where the class is struggling.
- **Watch the stuck points.** The dashboard flags any challenge with a solve rate under 35%. Now that briefs no longer contain the answer, that number means something: it marks a concept the class did not get, not a typing error.